

Lab 5 — Assignment 2: DynamoDB (NoSQL) CRUD Demo

Implement the Opposite Database Type to Assignment 1 (Mealie EC2 Application)

1. Architecture

Assignment 1 used Amazon RDS (SQL), so Assignment 2 uses the opposite type: Amazon DynamoDB (NoSQL). A new, lightweight Node.js/Express application was deployed on the same EC2 instance (mealie-lab4, i-0074ba1ec324ef939) alongside the existing Mealie application, sharing the same public IP and Caddy reverse proxy. Caddy routes by path: requests to "/" go to Mealie (127.0.0.1:9000, backed by RDS PostgreSQL), while requests to "/dynamo*" go to the new Node.js app (127.0.0.1:3000, backed by DynamoDB). No new EC2 instance or firewall rules were required.

2. Security: IAM Role (No Hardcoded Keys)

Per the mandatory security rule, DynamoDB access uses an IAM role attached directly to the EC2 instance, rather than hardcoded AWS access keys in code or configuration. The AWS SDK v3 automatically retrieves temporary credentials via the EC2 Instance Metadata Service (IMDSv2) at runtime.

Setting	Value
Role name	mealie-ec2-dynamodb-role
Attached policy	AmazonDynamoDBFullAccess
Attached to	EC2 instance mealie-lab4
Credentials in code	None (IMDSv2-based temporary credentials only)

3. DynamoDB Table

Setting	Value
Table name	mealie-dynamo-demo
Partition key	id (String)
Sort key	None
Capacity mode	On-demand

4. Schema / Datatype Proof (Mandatory: 5+ Datatypes)

Each item written to the table includes attributes spanning 5 distinct DynamoDB datatypes, satisfying the mandatory rule:

Attribute	DynamoDB Type	Example
id	String (S)	"item1"
name	String (S)	"Test Widget"
quantity	Number (N)	10
inStock	Boolean (BOOL)	true
tags	List (L)	["demo", "test"]

metadata	Map (M)	{"note": "first item"}
----------	---------	------------------------

5. Application

A minimal Node.js/Express application (/srv/dynamo-demo/app.js) using AWS SDK v3 (@aws-sdk/client-dynamodb, @aws-sdk/lib-dynamodb) exposes an HTML form and REST API for all CRUD operations. It runs as a systemd service (dynamo-demo.service), enabled for automatic startup on boot, consistent with the rest of the stack (Caddy, PostgreSQL, Docker/Mealie).

Method	Route	Operation
GET	/dynamo/	HTML UI
POST	/dynamo/api/items	Create
GET	/dynamo/api/items/:id	Read (single)
GET	/dynamo/api/items	Read (all / scan)
PATCH	/dynamo/api/items/:id	Update (increments quantity)
DELETE	/dynamo/api/items/:id	Delete

6. CRUD Demonstration

All four operations were demonstrated live through the running application at <http://<public-ip>/dynamo>:

Operation	Action performed
Create	Added item id=item1 with all 5 datatypes populated
Read	Retrieved item1 by ID, confirmed all attributes present
Update	Incremented quantity from 10 to 11
Delete	Removed item1, confirmed via subsequent scan

7. Evidence Summary

Screenshots captured: EC2 application running and reachable at its public IP under /dynamo; DynamoDB table showing status "Active" with the configured partition key; IAM role attached to the EC2 instance with no hardcoded credentials; and all four CRUD operations performed live in the browser, each showing the resulting JSON response with all 5 datatypes visible.